Integration Meeting Agenda — Eleven Contract Collisions

RESQ · Integration contract · 28 August 2026

Every place where two of us have already defined the same thing and meant different things. One decision each. Nothing here needs anyone to have read anyone else's code.

What this is not: a list of things one of us forgot. Those are additions, not arguments, and they don't need the room.

These eleven are different — each has two or three of us holding incompatible positions right now. Each needs one answer before any code merges.

Who's who

	MEMBER	OWNS
D	Faris	System core, engine, audit
C	Saif	Routing, graph, destinations
A	Huthaifa	Severity scoring
B	Layan	Dispatch — affected by items 4, 5, 6 and 8

---

1. What we call a call while it's being handled

- **D** — reported → queued → assigned → on scene → `transporting` → resolved / `failed`
- **C** — reported → queued → assigned → `in transit` → on scene → resolved

Options

- **A.** Use D's seven names. "In transit" is deleted — driving to the scene is already covered by "assigned".
- **B.** Name both journeys explicitly: rename "assigned" to "en route to scene", keep "transporting" for the hospital run.
- **C.** Use C's six names.

If we pick wrong. "In transit" and "transporting" describe opposite halves of a call — driving *to* the emergency versus driving a patient *away* to hospital. These names are plain text in both codebases, so a mismatch never produces an error; the comparison just quietly comes back false. The symptom is an audit screen claiming a patient was being transported during the minutes the crew was still driving to the scene — which is precisely the kind of thing we'll be asked to explain live. Separately, C's list has no name for a call that *couldn't* be served, so those calls have nowhere to end up.

Faris leans A. B is a defensible alternative but costs test rewrites for no new behaviour. C is not survivable.

---

2. What unit time is measured in

- **D** — simulation `seconds`
- **C** — simulation `ticks` (one tick = 30 seconds)
- **A** — `minutes`

Options

- **A.** Seconds everywhere, and the unit goes in the field name (`waiting_seconds`, not `waiting_time`).
- **B.** Ticks everywhere.
- **C.** Everyone keeps their own unit and converts at the boundary.

If we pick wrong. A tick is a **settings value**, not a unit. It's 30 seconds today. If anyone tunes it later, every timestamp already written to the database silently changes meaning — retroactively, across runs already recorded, with no error and every number still looking plausible. Option C means every pair of components needs a conversion that nobody owns. And

this one is live already: A's waiting-time calculation divides by 10 for minutes, so if it's handed seconds it maxes out after ten seconds of waiting and reports every call as maximally urgent.

Faris leans A. A's fix is one line — the reference constant becomes 600 and the label becomes seconds.

---

3. How severe a call is, and who gets to say — *my gap*

- **D** — one field for the *computed* score. **No field for what the caller reported** — my omission
- **C** — reported severity as `1–5`
- **A** — reads `Low/Medium/High/Critical`, returns `0–100` plus a category

Options

- **A.** Two fields: reported severity in A's four words, and A's 0–100 as the computed score.
- **B.** Two fields, but reported severity is C's 1–5 and A maps it on the way in.
- **C.** One field, overwritten with A's answer.

If we pick wrong. C and A each invented a field here because **the contract didn't have one** — it defined a slot for A's answer and none for the question. That's on me, and it's why there are three versions rather than two. Option C is the dangerous one: with a single field, A's score overwrites what the caller said, and we can no longer answer "was this ranked top because it was reported critical, or because it had been waiting twenty minutes?" That question is the whole point of explainable scoring. Note also that A's four levels and C's five don't divide evenly, so B needs a stated mapping rather than an assumed one.

Faris leans A. A's scale already exists and works; I add the missing input field.

---

4. What kinds of unit exist

- **D** — `AMBULANCE, FIRE_TRUCK, RESCUE_VAN, HAZMAT_TEAM`
- **C** — `AMBULANCE, FIRE, POLICE`

Options

- **A.** D's four.
- **B.** D's four plus police — needs an owner to add vehicles, a call type and a reason for them.
- **C.** C's three.

If we pick wrong. This one lands squarely on B. **Police currently has no vehicle anywhere in the city, no call type that asks for one, and no skill attached** — so dispatch logic written against it would match a unit that can never be selected, and nobody would notice until the demo. Going the other way, rescue vans and hazmat teams *do* exist in the city, so C's list strands real vehicles that no call can reach. And "fire" versus "fire truck" is a silent mismatch rather than an error — the comparison simply returns false.

Faris leans A. B is fine if someone genuinely wants police and will own it end to end.

---

5. What states a vehicle moves through

- **D** — available, en route, on scene, transporting, `returning`, out of service
- **C** — idle, en route, on scene, offline

Options

- **A.** D's six.
- **B.** C's four.

If we pick wrong. The two name differences are only spellings — idle/available, offline/out of service. The two *missing* states are real. With no "returning", **a crew driving back to base is either shown as free, so B dispatches a**

vehicle that isn't where the screen says it is, or shown as busy forever, which quietly ruins the utilisation figures we're comparing at the end. With no "transporting", the hospital run is invisible on the operator screen during the exact demo step it's meant to illustrate.

Faris leans A. The renames are free; the two additions are not optional.

---

6. Whether "what happened" and "what's needed" are one field or two

- **D** — two fields: kind of emergency, and kind of unit required
- **C** — one field: `required_unit_type`

Options

- **A.** Two fields.
- **B.** One field, with the unit type implying the emergency.

If we pick wrong. These answer different questions for different people: A scores urgency from *what happened*, B matches from *what's needed*. Collapse them and **A has to infer urgency by reading which vehicle is required**, which means A needs to know the fleet — that's B's territory, and it makes A's scoring harder to explain, not easier. It also can't express the case the brief opens with: a traffic collision needing both an ambulance and a fire truck.

Faris leans A. This one mostly costs A and B if we get it wrong, not me.

---

7. How hospital capacity is recorded

- **D** — total beds and beds in use are stored; free beds are worked out from them
- **C** — free beds are stored, and written directly by whoever wants to change them

Options

- **A.** Store two numbers, calculate the third.
- **B.** Store free beds and update it directly.

If we pick wrong. Two independently writable numbers that are supposed to agree will eventually stop agreeing — **a hospital showing three free beds out of a capacity of two** is the sort of thing that appears on screen mid-demo and can't be explained. Calculating one from the others makes that impossible rather than merely unlikely. It also changes what "hospital full" means during the demo: a state the simulation actually reaches, rather than a number someone typed in.

Faris leans A.

---

8. What a component does when it can't answer

- **D** — return a result carrying a named reason; never throw
- **A** — throws an error on a severity value it doesn't recognise
- **C** — no position stated

Options

- **A.** Named reasons at every boundary between components; validation errors stay inside the component that raised them.
- **B.** Throw, and let callers catch.
- **C.** Mixed — decided case by case.

If we pick wrong. "No unit free", "no route", "every hospital full", "a severity value I don't recognise" are all **normal events in this simulation, not programmer mistakes**. A thrown error at a component boundary stops the entire simulated minute rather than failing one call — the opposite of what the resilience part of the grade rewards, and it will

happen live, because the instructor is deliberately trying to break things. There's also nothing to show: a named reason renders on the audit screen, a stack trace doesn't.

Faris leans A. A keeps its validation exactly as written — it just doesn't escape the module.

---

9. Who's allowed to change the world when the instructor breaks something

- **D** — the control says *what to do*; the engine works out everything that changes
- **C** — the instruction carries the new state with it, e.g. "unit A3, status offline"

Options

- **A.** Controls express intent only.
- **B.** Controls carry the resulting state.

If we pick wrong. Taking a vehicle off the road **isn't one change** — the call it was handling has to go back into the queue for someone else. If the instruction sets the status directly, that second half is skipped, and a live emergency silently vanishes from the system while the screen still looks fine. The audit trail then describes a world that doesn't exist, which is the one thing it exists to prevent. Same shape for filling a hospital.

Faris leans A.

---

10. Whether a road closes one way or both

- **D** — each road is two one-way records, so a single direction can close
- **C** — one road, one open/closed flag; direction not addressed

Options

- **A.** Directional — closing a road means closing a direction, and closing both is two instructions.
- **B.** Whole-road — closing a road always closes both directions.

If we pick wrong. Because C's document doesn't say, **the same instruction produces two different worlds** depending on who implemented the reader — and routing results stop reproducing between runs, which quietly invalidates the before-and-after comparison at the end. Directional also makes a better demo: closing one direction forces a visible detour, while closing both can sever the map and the honest answer is just "no route", which shows nothing.

Faris leans A. Mainly C's call — he owns the routing that has to answer it.

---

11. Whether the contract is the document or the code

- **D** — the shared code files are the contract
- **C** — `docs/integration_contract.md` is the contract

Options

- **A.** Code is binding; the document describes it and names the files.
- **B.** Document is binding; the code is changed to match it.
- **C.** Keep both authoritative and sync them by hand.

If we pick wrong. Option C is what we've been doing, and **three days of it produced 59 differences across the two definitions** — the other ten items on this page are all symptoms of it. Whichever source wins, the other has to be explicitly demoted *and given an owner*, or we hold this meeting again in a fortnight. To be clear, this isn't an argument for dropping the document: a written data model is a required deliverable and we still have to produce one. The question is only which one is binding when they disagree.

Faris leans A, and I'll rewrite the document to match whatever we agree here.

Deliberately not on this page

Dependencies — the shared requirements file pins a web framework and a validation library the core doesn't use. Real disagreement, but it's a build decision, not a contract collision.

Folder ownership — `src/engine/` currently holds routing, dispatch scoring and audit together, which is three owners in one place. Being handled separately.

Also not on this page

Roughly three dozen places where only one of us defined something and the others were silent — capabilities, transport flags, route objects, the timestamps behind the audit screen. Those are additions, not arguments, and they don't need the room. The full field-by-field comparison in `docs/contract_comparison.md` has them.

Nothing has been merged or committed. These eleven answers unblock that.